

PROJETO ROBÔ HEXÁPODE PARA SEGURANÇA PÚBLICA

MVP 12 Meses — Orçamento Enxuto

Equipe Voluntária | Foco em Componentes Essenciais

Versão: 5.0 | Junho de 2026

SUMÁRIO

- I. Introdução e Definição de MVP
- II. Visão Realista
- III. Cronograma Executivo (12 Meses)
- IV. Divisão de Trabalho e Competências
- V. Orçamento Realista
- VI. Recursos Gratuitos
- VII. Riscos e Mitigação
- VIII. Documentação Esperada
- IX. Timeline Visual
- X. O que a Equipe Vai Aprender
- XI. Próximas Ações
- XII. Definição de Sucesso em 12 Meses
- XIII. Referências de Projetos Hexápodes

I. INTRODUÇÃO E DEFINIÇÃO DE MVP

O Projeto

O ArachnoBot é uma proposta de Veículo Terrestre Não Tripulado (UGV) com morfologia hexápode, desenvolvido por equipe voluntária com objetivo de demonstrar, em 12 meses, um protótipo funcional capaz de locomover-se em terrenos variados, identificar pessoas por meio de visão computacional e ser operado remotamente via Wi-Fi. O projeto situa-se na interseção de robótica móvel, aprendizado de máquina e sistemas embarcados, com ênfase em baixo custo e tecnologia aberta.

O que é MVP

MVP — Minimum Viable Product, ou Produto Mínimo Viável — é o menor conjunto de funcionalidades que permite validar o conceito central de um projeto com o menor desperdício possível de tempo, dinheiro e esforço. A ideia vem do desenvolvimento de software, mas se aplica diretamente a projetos de hardware e robótica: em vez de tentar construir o robô definitivo de uma vez, constrói-se a versão mais simples que ainda demonstra que o conceito funciona.

No contexto deste projeto, o MVP não é o robô ideal — é o robô mínimo que anda, enxerga e obedece. Qualquer funcionalidade além disso (navegação autônoma completa, mapeamento 3D, comunicação de longo alcance, payload de resgate) pertence a versões futuras. Essa disciplina é essencial para uma equipe voluntária com horas limitadas: sem um escopo mínimo bem definido, o projeto tende a crescer indefinidamente e nunca terminar.

O MVP deste projeto é considerado alcançado quando o robô: locomove-se de forma estável em múltiplos terrenos, transmite vídeo ao vivo para o operador, detecta pessoas com modelo de machine learning embarcado no Raspberry Pi, opera com autonomia mínima de uma hora e pode ser controlado remotamente via Wi-Fi — tudo isso dentro do orçamento de R\$ 700 a R\$ 920 em componentes novos e em 12 meses de trabalho.

II. VISÃO REALISTA

Objetivo

Entregar um protótipo funcional de robô hexápode que demonstre: locomoção em terrenos variados, visão computacional com detecção de pessoas, autonomia básica e aprendizado prático em engenharia mecânica, eletrônica e machine learning.

Chassi e Estrutura

O chassi e todas as peças estruturais serão produzidos em impressora 3D Creality K1C com filamento PLA. O projeto não parte do zero: já existem modelos CAD prontos de robôs hexápodes open source como referência base, que serão adaptados conforme as especificidades da equipe. Isso elimina semanas de modelagem do zero e permite focar na integração dos sistemas.

Equipe

Voluntários entre 8 e 12 pessoas trabalhando em horas livres, sem vínculo formal. Todos os integrantes compartilham a responsabilidade de testes e documentação ao longo do projeto — não existe uma pessoa dedicada exclusivamente a essa função.

Orçamento

MVP Mínimo: R\$ 700. MVP com sensores básicos: R\$ 920. A equipe já possui: Raspberry Pi 4B, câmera RPi v2, bateria Li-Po 11.1V, ESP32 e filamento PLA — totalizando aproximadamente R\$ 880 em componentes disponíveis.

Tempo por Pessoa

Aproximadamente 8 a 12 horas semanais por integrante, compatível com rotina de trabalho ou estudo.

III. CRONOGRAMA EXECUTIVO (12 MESES)

T1 — Meses 1 a 3: Design e Prototipagem

- Mês 1: Adaptação do modelo CAD de referência para a K1C, definição de encaixes e tolerâncias de impressão
- Mês 2: Compra dos componentes (R\$ 700), início das impressões, montagem dos servomotores MG90 na primeira perna
- Mês 3: Uma perna funcional com controle de servos validado

T2 — Meses 4 a 6: Integração e Locomoção

- Mês 4: Seis pernas montadas e controladas pelo firmware base
- Mês 5: Algoritmos de marcha (tripod gait) implementados e testados em campo
- Mês 6: Câmera integrada com stream de vídeo ao vivo, pipeline de detecção de pessoas ativo

T3 — Meses 7 a 9: Machine Learning e Autonomia

- Mês 7: Coleta do dataset (500+ imagens), treinamento inicial no Google Colab
- Mês 8: Modelo ML integrado ao Raspberry Pi, inferência em tempo real
- Mês 9: Autonomia básica funcionando — robô reage a detecção sem intervenção do operador

T4 — Meses 10 a 12: Testes, Polimento e Documentação

- Mês 10: Testes em campo com diferentes terrenos e condições de luz
- Mês 11: Correções, otimização de consumo de bateria, polimento geral
- Mês 12: Documentação final, vídeo de demonstração, relatório de lições aprendidas

IV. DIVISÃO DE TRABALHO E COMPETÊNCIAS

A tabela abaixo apresenta as funções, quantidade de integrantes e carga horária semanal estimada. Testes e documentação são responsabilidade compartilhada de toda a equipe — não existe função dedicada para isso.

Função	Qtd	Horas/semana	Área Principal
Lead Mecânico	1	10h	Engenharia Mecânica
Mecânico	1	8h	Engenharia Mecânica
Lead Eletrônico	1	10h	Engenharia Elétrica
Firmware Lead	2	12h	Sistemas Embarcados / Integração
Firmware Dev	1	8h	Sistemas Embarcados
ML Engineer	1	10h	Machine Learning
Visão Computacional	1	8h	Computer Vision

Tabela 1 — Composição da equipe e carga horária

Lead Mecânico

O Lead Mecânico é o responsável central pelo projeto estrutural do robô. Adapta os modelos CAD de referência para a impressora K1C, define tolerâncias de encaixe considerando as características do PLA e as dimensões dos servomotores MG90, revisa e aprova cada peça antes da impressão, decide estratégias de reforço estrutural nas regiões de maior esforço mecânico (juntas de quadril e fêmur), e resolve problemas de montagem que exijam redesenho. É o ponto final de decisão para qualquer questão de estrutura física.

Mecânico (1 pessoa)

Com modelos CAD de referência prontos e a K1C disponível, uma única pessoa de bancada é suficiente para este escopo. O Mecânico opera a K1C para impressão das peças, realiza o pós-processamento (remoção de suportes, lixamento de encaixes), monta as pernas e o chassi, aperta os parafusos nos torques corretos para não comprimir o PLA, e testa a amplitude de movimento de cada articulação antes de passar para a eletrônica. Todo problema que exija redesenho vai de volta ao Lead Mecânico.

Lead Eletrônico

O Lead Eletrônico é responsável por todo o projeto de circuito do robô: projetar o esquema elétrico completo da distribuição de energia (bateria Li-Po 11.1V → BEC 5V/3A → servos e ESP32), definir a topologia de conexão dos 18 servomotores MG90 evitando sobrecarga de corrente em qualquer ponto, projetar ou montar a PCB, escolher e validar conectores XT60 e JST, e garantir que nenhum componente opere fora da faixa segura de tensão ou corrente. É o responsável final por qualquer decisão de circuito e pela segurança elétrica do sistema.

Firmware Lead (2 pessoas)

Dada a ausência de especialistas em controle e na integração dos nós do projeto, a função é ocupada por duas pessoas com divisão interna clara. O primeiro Firmware Lead cuida da arquitetura de software embarcado e controle: define como o ESP32 gera sinais PWM para os 18 servos em paralelo sem conflito de threads, estabelece o protocolo de comunicação serial/I2C entre ESP32 e Raspberry Pi, escreve o código de controle de marcha em C (tripod gait) e integra o driver do IMU MPU6050. O segundo Firmware Lead cuida da infraestrutura de integração e DevOps: configura e mantém o servidor de CI (container Docker rodando no próprio Raspberry Pi ou em servidor externo), gerencia o repositório GitHub com branches protegidas e pipelines de CI/CD, estabelece a comunicação de telemetria Wi-Fi (protocolo, autenticação e stream de dados), e garante que os módulos de ML e visão computacional se integrem corretamente ao sistema operacional do Pi. Ambos compartilham o debugging de integração entre áreas.

Firmware Dev

O Firmware Dev implementa funcionalidades dentro da arquitetura definida pelos dois Firmware Leads — sem responsabilidade de decisão de arquitetura ou DevOps. Na prática: escreve as rotinas de PWM, o driver de comunicação I2C com o IMU MPU6050, o protocolo de recebimento de comandos via Wi-Fi e os loops de controle dos servos. A diferença central em relação aos Leads é que o Dev executa dentro das regras que os Leads estabeleceram, e reporta a eles quando encontra limitações de arquitetura que precisam ser revisadas.

ML Engineer

O ML Engineer é responsável por todo o pipeline de aprendizado de máquina: define a estratégia de coleta do dataset (ambientes, iluminação, ângulos), organiza e anota as imagens (mínimo 500), escolhe a arquitetura de rede neural mais leve para inferência no Raspberry Pi (MobileNet, YOLOv5n ou similar), treina o modelo no Google Colab com transfer learning a partir de pesos pré-treinados, exporta o modelo em formato compatível com o Pi (TensorFlow Lite ou ONNX), e retreina quando o modelo não generalizar bem para novos ambientes.

Visão Computacional

A Visão Computacional integra o modelo do ML Engineer com a câmera RPi v2 em tempo real: escreve o pipeline de captura de frames com OpenCV, passa cada frame pelo modelo de detecção e extrai bounding boxes com confiança mínima configurável, implementa tracking para manter o ID de cada pessoa detectada entre frames, transforma as saídas de detecção em sinais de controle que o firmware usa (ex.: pessoa à esquerda → virar esquerda), e otimiza o pipeline para atingir taxa mínima de 10 FPS no Pi sem travar o sistema.

Testes e Documentação — Responsabilidade Compartilhada

Não existe uma pessoa dedicada a testes e documentação. Cada integrante documenta o que produz: o Lead Mecânico entrega os arquivos STL e CAD final; o Lead Eletrônico entrega o esquema elétrico e BOM com links; os Firmware Leads mantêm o README técnico e o histórico do repositório GitHub; o ML Engineer documenta o processo de treinamento e métricas; e a Visão Computacional registra os parâmetros do pipeline. Nos meses 10 a 12 toda a equipe contribui para o vídeo final, o relatório de lições aprendidas e o guia de montagem.

V. ORÇAMENTO REALISTA

Opção 1: MVP Mínimo — R\$ 700

Item	Qtd	R\$
Servomotores MG90	18	324
Alumínio (barras 20x20, 1m total)	—	120
Parafusos, porcas, rolamentos (kit)	—	100
Fios, conectores, garras servo	—	80
PCB (perftape ou fenolite)	—	40
BEC 5V/3A	—	30
Conectores XT60, JST (kit)	—	6
TOTAL	—	700

Tabela 2 — Componentes MVP mínimo

Opção 2: MVP com Sensores Básicos — R\$ 920

Item	Qtd	R\$
Opção 1 (base)	—	700
Sensor Ultrassom HC-SR04	4	120
IMU MPU6050	1	40
Extras (LEDs, buzzer, fita)	—	60
TOTAL	—	920

Tabela 3 — Componentes MVP com sensores

Componentes Já Disponíveis na Equipe

Item	Valor estimado (R\$)
Raspberry Pi 4B	350
Câmera RPi v2	150

Item	Valor estimado (R\$)
Bateria Li-Po 11.1V	150
ESP32	80
Filamento PLA (K1C)	150
Impressora K1C	já disponível
SUBTOTAL disponível	~R\$ 880

Tabela 4 — Componentes disponíveis

Total geral do projeto considerando componentes novos mais os já disponíveis: entre R\$ 1.580 e R\$ 1.800.

VI. RECURSOS GRATUITOS

Recurso	Plataforma	Custo
CAD	Fusion 360 (licença educação)	Grátis
Impressão 3D	K1C (já disponível)	Grátis
GPU para ML	Google Colab	Grátis (15h/mês)
Repositório e CI	GitHub + GitHub Actions	Grátis
Containers	Docker (no próprio Pi)	Grátis
Comunicação	Discord / Telegram	Grátis
Documentação	Google Docs / Notion	Grátis
Dev Tools	VS Code, Python, TensorFlow	Grátis (open source)

Tabela 5 — Recursos disponíveis sem custo

VII. RISCOS E MITIGAÇÃO

Servo MG90 fraco

Testar 2 a 3 unidades antes de comprar as 18. Se insuficiente, usar redução mecânica impressa em PLA ou migrar para MG996R no eixo de quadril.

Peça impressa quebrando

Imprimir com 40% de preenchimento mínimo e pelo menos 3 paredes. Manter STLs versionados no GitHub para reimprimir qualquer peça rapidamente.

Cinemática não convergindo

Testar primeiro em simulação (CAD animado). Se não funcionar, usar 2 graus de liberdade por perna no lugar de 3.

Bateria descarregando rápido

Normal com 18 servos em carga. Objetivo de autonomia: 1 a 2 horas. Monitorar consumo com medidor de corrente e identificar servos que travam.

ML não generalizando

Coletar mais dados com variação de iluminação e ângulo. Retreinar no Colab. Usar data augmentation para ampliar o dataset sem coletar mais imagens.

Integração entre módulos falhando

Os dois Firmware Leads são responsáveis por integração. Estabelecer interfaces definidas entre módulos antes de começar — ninguém começa sem documentar a interface que vai exportar.

Docker ou servidor de CI caindo

O Raspberry Pi do próprio robô pode hospedar o servidor de integração em container. Se o Pi estiver em campo, usar GitHub Actions como fallback de CI.

Equipe desistindo

Equipe de 8 pessoas reduz o risco de paralisia por saída de um integrante. Documentação clara no GitHub garante que o substituto consiga continuar sem depender de transferência de conhecimento verbal.

Demora no AliExpress

Comprar os componentes 1 a 2 meses antes do início da montagem. Peças pequenas (parafusos, conectores) comprar em loja local como backup.

VIII. DOCUMENTAÇÃO ESPERADA (MÊS 12)

1. Robô Funcional

- Protótipo v1.0 construído, testado e demonstrável publicamente
- Anda em múltiplos terrenos (concreto, grama, terreno irregular leve)
- Detecta pessoas com ML em tempo real
- Autonomia mínima de 1 hora

2. Código Fonte (GitHub)

- Firmware em Python e C, mínimo 3.000 linhas de código
- Modelos ML exportados em TensorFlow Lite ou ONNX
- Dataset anotado com mínimo de 500 imagens
- README claro com instruções de build, flash e operação
- Dockerfile para o ambiente de integração

3. Documentação Técnica

- Guia de montagem passo a passo com fotos
- Arquivos CAD (Fusion 360) e STLs prontos para impressão na K1C
- Bill of Materials (BOM) com links e preços atualizados
- Esquema elétrico completo
- Guia de operação do robô

4. Mídia

- Vídeos de testes em campo
- Vídeo final de demonstração (3 a 5 minutos)
- Fotos do processo de construção
- Slides para apresentação

5. Relatório Final

- O que funcionou e por quê
- O que não funcionou e por quê
- Lições aprendidas por área
- Roadmap para a versão 2.0

IX. TIMELINE VISUAL

Período	Marcos
Meses 1–3	CAD adaptado Componentes comprados 1 perna funcional
Meses 4–6	6 pernas montadas Algoritmos de marcha Câmera + detecção
Meses 7–9	Dataset + treinamento ML no robô Autonomia básica
Meses 10–12	Testes em campo Polimento Documentação final

Tabela 6 — Timeline executivo

X. O QUE A EQUIPE VAI APRENDER

Área	Conteúdo
Mecânica	Cinemática, dinâmica de hexápodes, impressão 3D com K1C, tolerâncias de PLA
Eletrônica	PCB design, distribuição de potência, seleção de componentes
Firmware	C embarcado, PWM, I2C, threads, protocolos de comunicação serial e Wi-Fi
Integração / DevOps	Docker, GitHub Actions, CI/CD, integração hardware-software, debugging de sistema
Machine Learning	Transfer learning, dataset curation, treinamento no Colab, exportação de modelos
Visão Computacional	OpenCV, CNN, detecção de objetos, tracking, pipeline em tempo real

Tabela 7 — Aprendizados por área

XI. PRÓXIMAS AÇÕES

Esta semana

- Reunir a equipe e confirmar os integrantes de cada função
- Definir responsabilidades individuais e os dois Firmware Leads
- Montar cronograma compartilhado no GitHub Projects ou Notion
- Pesquisar e selecionar o modelo CAD de referência para adaptação

Próximas 2 semanas

- Iniciar adaptação do CAD no Fusion 360
- Fechar lista de compra (BOM) e realizar o pedido dos componentes
- Criar o repositório GitHub com estrutura de branches, README inicial e Dockerfile base
- Configurar o ambiente Docker no Raspberry Pi

Mês 1

- CAD da primeira perna finalizado e aprovado pelo Lead Mecânico
- Componentes recebidos e conferidos
- Primeira perna impressa na K1C e montada

XII. DEFINIÇÃO DE SUCESSO EM 12 MESES

O projeto será considerado bem-sucedido quando entregar um protótipo funcional que atenda aos seguintes critérios:

- Anda em múltiplos terrenos de forma estável
- Transmite vídeo ao vivo para o operador
- Detecta pessoas com ML embarcado no Raspberry Pi
- Opera com autonomia mínima de 1 hora
- Pode ser controlado remotamente via Wi-Fi
- Está completamente documentado no GitHub
- Pronto para demonstração pública
- Serve como base sólida para a versão 2.0

Custo em componentes novos: R\$ 700 a R\$ 920

Tempo total: 12 meses com 8 a 12 horas semanais por pessoa

Equipe: 8 voluntários

Impacto: portfólio técnico relevante para currículo e GitHub de todos os integrantes

XIII. REFERÊNCIAS DE PROJETOS HEXÁPODES

Os repositórios abaixo servem como referência de arquitetura, código e documentação. Os modelos CAD e algoritmos devem ser adaptados e não copiados diretamente. Dar crédito quando utilizar código ou estruturas de terceiros.

Repositório	Relevância
mithi/hexapod	Cinemática inversa e simulação visual — base matemática do movimento
MakeYourPet/hexapod	Design mecânico acessível e controle de servos — referência de montagem
rookidroid/hexapod	Implementação embarcada de baixo custo — referência de firmware
SmallpTsai/7688-hexapod	Integração com IoT e comunicação Wi-Fi — referência de telemetria
neuroprod/InsectRobotSimulation	Simulação de locomoção bioinspirada — validação de algoritmos
ByteRyze/Hexapod_2	Referência de hardware e estrutura física — tolerâncias e encaixes
ClimbSnail/HexapodRobot_STC15	Firmware em microcontrolador de baixo custo — referência de C embarcado
JakobLeander/hexapod	Planejamento de trajetória e controle de marcha — algoritmos de gait

Tabela 8 — Repositórios de referência